Evaluating Checks for Generated KPI Documentation: Rule Coverage and Reference Criteria

Meshaal Obaid Mouawad[a,*], Bryan Jones[a]

[a]Department of Electrical and Computer Engineering, Mississippi State University, Mississippi State, 39762, MS, USA

ARTICLE INFO

Keywords:
software documentation
empirical evaluation
program comprehension
checking rules
reference criteria
key performance indicators

ABSTRACT

Evaluating a documentation checker requires agreement about the property being checked and the unit counted as an error. We investigate these requirements using 28 generated key performance indicator (KPI) documents, their linked source code, 32 checker controls, and 336 categorical assessments from three code reviewers. The checker returned every expected control decision. Its source-role rule flagged two of the 16 documents assigned Fail by the reviewer majority, yielding 12.5% recall against those document labels. Inspection of the recorded rationales showed that, in both overlapping documents, the majority reviewers and checker identified different problems. Conversely, two alerts on majority-Pass documents identified concrete source-role errors. The source-role comparison therefore exposed differences between implemented patterns and the properties assessed under the same criterion. A second comparison examined ten documents with eligible calculation conditions. The same five checker alerts had precision of 40% against the identical labels of reviewers A and B and 100% against reviewer C's labels. Their rationales differed in both the condition assessed and the observations about its explanation. We connect these results to source excerpts, generated descriptions, reviewer rationales, and generator code. The study contributes an empirical account of how rule scope, document-level aggregation, and reference interpretation shape checking results, together with a reproducible case ledger for distinguishing these mechanisms.

1. Introduction

Automated checks for generated software documentation can follow their specified rules and still disagree with reviewers about the documents they assess. Understanding that disagreement requires examining both the checker and the reference judgments. This study investigates that relationship in 28 source-linked key performance indicator (KPI) documents. All 32 prepared control outcomes were reproduced, yet the checker and reviewer majority agreed on failure for only two of 16 source-role documents. Inspection of the rationales showed that even those matching labels referred to different problems. The contribution is a source-grounded explanation of how rule scope and reference meaning determine the interpretation of checking results.

A KPI connects an organizational measure to the calculation producing its value. Its documentation may bring together a business formula, source excerpts, descriptions of those excerpts, safeguards for exceptional inputs, and a review status. These fields make a calculation inspectable. They also create several relationships to check: whether a source description assigns the correct programming role, whether the formula matches the implementation, and whether the written explanation states conditions that change the returned value. Research on documentation defects and code–comment inconsistencies establishes the importance of inspecting these relationships (Aghajani et al., 2019; Wen et al., 2019; Zhou et al., 2017).

*Corresponding author: Meshaal Obaid Mouawad
Email addresses: mm4922@msstate.edu (M.O. Mouawad); bjones@ece.msstate.edu (B. Jones)
ORCID(s): 0000-0002-1152-8324 (M.O. Mouawad); 0000-0002-5807-8797 (B. Jones)

We study documents generated by LEAP, short for Literate Programming for Automated KPI Extraction. In the evaluated version, LEAP scans source files for KPI candidates and publishes an individual HTML document for each selected calculation. The collection is called the Interactive Bluebook. The document's *Developer Lineage* field explains the role of a source excerpt; its *Formal Formula* shows the calculation; and its *Logic Safeguards* field describes exceptional-input handling. A separate checker compares selected source and document fields using deterministic rules. Three code reviewers assessed the same saved documents without seeing the checker outputs.

The Ethylene Production Yield document provides a concrete example. The retained Python function returns zero when feedstock input is zero or negative. Its written safeguard omits that branch. Two reviewers assigned condition Pass, whereas the third reviewer and checker assigned Fail. One Pass rationale states that the guard is explicitly safeguarded, in conflict with the preserved explanatory field. This makes the relationship between a label, its rationale, and the document evidence directly inspectable. A second error on this document concerns a source role: the generator calls the word "a" in a function docstring an output alias, a programming name assigned to a result. The source-role checker and one reviewer flag that error, while the majority assigns Pass.

These observations connect two established research concerns. Documentation-verification methods define particular properties to check (Kuhn and Bergel, 2014; Lozano et al., 2015), and empirical evaluations examine which real problems those methods find (Habib and Pradel, 2018). Documentation-to-code equivalence additionally makes behavioral completeness an explicit target (Eghbali et al., 2026). Our question concerns evaluation when a generated document

combines several explanatory fields and reviewers apply a criterion more broadly, or to a different publication layer, than its automated implementation.

We answer that question by preserving the original control outcomes and document-level comparisons, then inspecting the source and rationales behind agreements and disagreements. The source-role analysis covers all 16 majority-Fail documents and all four checker alerts. The condition analysis compares unchanged alerts against each reviewer's recorded labels and examines the accompanying rationales. Historical and alternative-input outputs provide further evidence about the generator mechanisms associated with the observed descriptions.

The study contributes three connected results: a case-level account of how source-role judgments exceed or differ from the implemented patterns; evidence that matching document labels can conceal different defect rationales; and an exact comparison showing how recorded reference judgments change support for the same alerts. The accompanying source/document/rationale ledger makes these distinctions inspectable and gives subsequent checker evaluations a concrete basis for defining their comparison unit and reference criteria.

2. Related Work

2.1. Making source-derived documentation checkable

Source-connected documentation has a long history. Literate programming connects program explanation and executable structure (Knuth, 1984). For documentation derived from existing systems, van Deursen and Moonen (2006) used inferred types to expose abstractions and dependencies in COBOL programs. Their TypeExplorer study demonstrates how generated views can communicate relationships beyond source cross-references. This provides context for LEAP's combination of code, formulas, and explanatory fields: the generated representation must communicate the intended relationship accurately.

Kuhn and Bergel (2014) investigated whether internal software documentation could be stated in controlled natural language and verified against a source model. Their approach combines controlled English, extracted code facts, reasoning, and queries. A software case study and a 20-participant experiment examined its feasibility and comprehensibility. The formal interpretation of the documentation determines which statements the reasoner can assess. In a complementary approach, Lozano et al. (2015) encoded structural source-code regularities as usage contracts and evaluated them in an industrial application. Their work shows how an explicit vocabulary makes implicit expectations executable.

These studies establish methods for checking precisely stated documentation or structural properties. Our study examines the evaluation of rules applied to explanations already generated across several fields. The relevant distinction is between the property a rule implements and the property a reviewer understands a criterion to request. The recorded rationale for a decision is therefore part of the comparison evidence.

2.2. Detecting inaccuracies and explaining behavior

Documentation-error research addresses several related targets. Zhong and Su (2013) detected inconsistent references to application programming interface (API) elements and code examples. Zhou et al. (2017) compared API documentation and code to identify defects in parameter constraints and exception directives. These methods make particular relationships between text and implementation checkable. Our source-role and condition checks likewise require an explicit comparison: a source excerpt and its assigned role, or an implemented condition and its written safeguard.

Empirical studies explain the variety of relationships that can fail. Wen et al. (2019) studied code–comment changes across Java projects and manually classified repaired inconsistencies. Aghajani et al. (2019) examined documentation-related discussions and reports to characterize a broader range of problems. Their findings motivate inspecting the nature of a disagreement rather than treating every Fail label as the same kind of documentation error. Xu et al. (2024) further addressed imperfect labels in learned code–comment inconsistency detection through confidence-based learning. In the present study, the label issue is directly inspectable: reviewers sometimes discuss ownership, formula meaning, or source roles under the same criterion.

Generated explanations also raise the question of which behavior must be communicated. Yang et al. (2026) translated Object Constraint Language contracts into natural-language descriptions and assessed their informational content. Eghbali et al. (2026) defined documentation-to-code equivalence and used code reconstruction to generate and evaluate behaviorally informative documentation. These contributions establish semantic content and completeness as research targets. We examine how a narrower condition check compares with reviewer judgments about the corresponding source branch and explanatory field.

2.3. Interpreting evaluation results

A checker evaluation needs evidence about both detected and undetected cases. Habib and Pradel (2018) applied three static bug detectors to 594 known bugs and inspected candidate warning–bug matches. Their missed-bug analysis distinguished problems related to existing patterns from problems outside those patterns. Matching the particular defect is central to that evaluation. For our document-level labels, two assessments may both report Fail while referring to different source fragments or fields. We therefore inspect rationales after reproducing the categorical comparison.

Evaluation meaning also matters for generated text. Roy et al. (2021) examined whether automatic code-summary metrics reflect human assessments, while Shi et al. (2022) investigated the effects of metric implementations, preprocessing, and datasets on summarization evaluation. These studies show why an apparent score must be interpreted through its measurement procedure. Our comparison concerns binary checker alerts whose reference labels differ.

Table 1
Closest published work and the distinct comparison examined here. The studies use different tasks and datasets; their numerical performance measures are not combined.

Published work	Established research contribution	Relationship to this study
Kuhn and Bergel (2014)	Controlled-language documentation verified against a source model	Examine generated explanations and the scope of the rules used to assess them.
Lozano et al. (2015)	Executable contracts for structural regularities	Make the rule's supported distinctions explicit before interpreting reviewer comparisons.
Zhou et al. (2017)	Detection of parameter and exception directive defects	Compare a source condition with the publication field required to disclose it.
Habib and Pradel (2018)	Real-bug coverage with inspected warning–bug matches	Distinguish a matching document label from agreement about a specific defect.
Wen et al. (2019)	Empirical taxonomy of code–comment inconsistencies	Inspect the relationships underlying Fail judgments rather than pooling their meanings.
Eghbali et al. (2026)	Behavioral documentation equivalence and generation	Inspect recorded judgments of a narrower disclosure requirement.
Roy et al. (2021); Shi et al. (2022)	Evaluation-metric and implementation effects in code summarization	Explain criterion-dependent support for unchanged checker alerts.

Inspecting the recorded reasons establishes which conditions and explanatory fields each assessment addresses.

Table 1 summarizes the closest relationships. The study connects established verification and evaluation questions through a fixed generated-document corpus with preserved source evidence and reviewer rationales. The resulting contribution is the diagnosis of evaluation disagreement in that corpus and the evidence structure used to examine it.

3. Research Questions and Method

The study is an archival case study of a fixed generator output and its checking procedure. It combines source/document inspection, conformance tests, three reviewers' recorded assessments, and a retrospective analysis of the rationales accompanying those assessments.

RQ1 Which source/document discrepancies are produced by the evaluated generator, and which recur across its archived and alternative-input outputs?

RQ2 How consistently do reviewers apply the four proposed criteria, and how do checker decisions compare with their document-level labels?

RQ3 Which differences in rule scope and reviewer rationale explain source-role agreements and disagreements?

RQ4 How does support for unchanged condition alerts vary across the recorded reviewer references, and what do their rationales explain?

RQ1 connects source excerpts, generated descriptions, and generator code. RQ2 reproduces the categorical comparisons. RQ3 inspects the reasons behind the source-role labels, including matching labels. RQ4 holds the eligible documents and checker output fixed while comparing the recorded reviewer references. Table 2 links each question to its evidence and unit.

3.1. Artifacts and selection

The primary dataset contains all 28 HTML KPI documents from the preserved evaluation build. It includes 9 documents labeled *Validated* and 19 labeled *Needs Review* by LEAP. These are generator-recorded statuses. Every checker and reviewer assessed the same saved documents. The preserved hash record identifies an artifact copy at commit `5e697b5b` whose 28 files match the primary documents byte-for-byte; full identifiers and SHA-256 hashes are retained in the technical supplement.

The earlier archived output contains 18 documents, including 6 *Validated* and 12 *Needs Review*. Two further input configurations used the same generator: 38 scenario-fixture files produced 29 documents, and 8 challenge files produced 8 documents. These artifacts support comparisons across states and input configurations of the evaluated implementation. The generation log records deterministic detail preparation for all 28 primary documents.

A separate adapter exercise used four official FastAPI tutorial sources and their generated OpenAPI 3.1 descriptions at pinned upstream identities (FastAPI Project, 2026; OpenAPI Initiative, 2025). FastAPI is a Python framework for application programming interfaces; OpenAPI describes an interface's inputs and constraints. An adapter maps the source and published fields of that format into the checker's expected inputs. This exercise records whether the checks execute on a second documentation format.

3.2. Criteria and implemented rules

The study considered four assessment criteria. Each was expressed as a reviewer question and implemented by a rule that recognized specified evidence patterns. We distinguish the *criterion*, the property being judged, from the *rule*, its implemented decision procedure.

Source-role accuracy. A source element's role describes what it does: a comment explains code, a conversion changes data representation, and an alias names an output. The checker

Table 2
Research questions, evidence, and analysis units. A rating is one reviewer's category for one document and criterion.

Question	Evidence	Unit	Analysis
RQ1	28 primary, 18 historical, 29 scenario, and 8 challenge documents; retained sources and generator excerpts	Document, source excerpt, eligible guard	Description/source comparison and code inspection
RQ2	32 expected control decisions; 336 ratings across 112 document–criterion combinations	Control decision; rating; document label	Conformance, agreement, majority comparison
RQ3	All 16 source-role majority-Fail documents and all 4 checker alerts, with rationales	Source/description/rationale relationship	Rule-scope and same-defect inspection
RQ4	Ten checker-eligible documents and the three reviewers' condition labels	Document under a named reviewer reference	Two confusion matrices for the same five alerts

compares a source excerpt with its Developer Lineage description. It flags comments or docstrings described using selected executable-role labels, and Structured Query Language (SQL) conversion types described as output aliases. For example, in `CAST(... AS FLOAT) AS on_spec_percentage`, `FLOAT` is a conversion type and `on_spec_percentage` is the output alias. Figure 1 shows the generated misdescription.

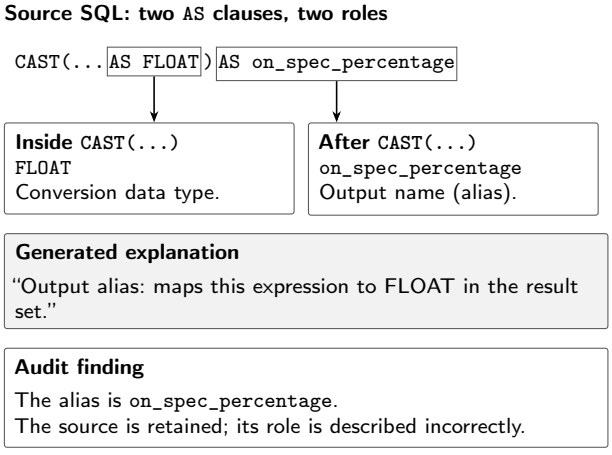


Figure 1: On-Spec Rate source-role error (C27). The generated description names the SQL conversion type as the output alias. The checker and Code reviewer C flag this error; Code reviewers A and B assign source-role Pass.

Condition disclosure. A *guard* is an explicit condition that changes a calculation for particular inputs. The rule recognizes denominator guards expressed through `NULLIF(variable,0)` or an `if`-family comparison with zero. It requires the safeguard field to state the zero or non-positive behavior and connect the guarded variable to the denominator of the displayed formula. A non-positive comparison requires wording that covers zero and negative inputs. The primary scanner found ten eligible documents.

Representation disagreement. This criterion concerns differences between recorded descriptions of a calculation. The implemented rule recognizes explicit conflict, mismatch, or inversion markers. It checks that the document remains marked *Needs Review* and appears in the Review Queue, the list of items requiring a decision. We analyze this explicit-marker routing outcome separately from the reviewers' broader disagreement judgments.

Review-authority consistency. This criterion concerns the relationship between unresolved status and certification-oriented wording. The implemented rule looks for specified phrases, including "certified metric dossier," "certified calculation expression," and "print certified dossier," on documents marked *Needs Review*. These are original interface strings. The reviewer exercise measures how consistently the broader criterion was applied.

3.3. Checker controls and source inspection

Conformance tests compare the checker's decisions with expected outcomes specified for prepared inputs. Eighteen rule tests cover source roles, denominator and safeguard information, routing, certification wording, and applicability. Fourteen artifact-derived controls comprise five unchanged baselines, eight substitutions of extracted inputs, and one deliberately modified HTML document read again through the adapter. Expected labels were stored separately from the checker outputs. The 32 expected outcomes comprise 19 Fail, 11 Pass, and two N/A decisions, where N/A means not applicable.

The same released rule library supports the primary, historical, alternative-input, and external-format checks. The technical scripts record document counts and identities, source-role findings, guard inventories, status and conflict outcomes, and control results. After locating discrepancies, we inspect the archived generator code and record the transformation associated with each generated description. Repeated descriptions are linked to a shared mechanism when the preserved code explains their occurrence across the corresponding documents.

3.4. Reviewer procedure and categorical analysis

Three code reviewers, identified as A, B, and C, assessed cases C01–C28. Phase A asked each reviewer to identify issues using their own judgment. Phase B asked for Pass, Fail, or N/A judgments with evidence for each of the four criteria. The packet contained both phases and directed reviewers to complete Phase A first. It excluded checker

outputs, manuscript result tables, the organizer's case-to-filename mapping, and the other reviewers' responses.

All 28 Phase-A case rows were returned by each reviewer. Phase B contains 112 document–criterion combinations and three ratings per combination, giving 336 ratings. The preserved normalized analysis dataset contains 242 Pass/Fail ratings and 94 N/A ratings: 40 of the 84 condition ratings and 54 of the 84 disagreement ratings. The associated intermediate export represents those 94 positions as blanks. Cell-by-cell reconciliation with the three preserved original responses reproduced all 336 labels, including the 94 explicit N/A values. Reviewer-specific N/A counts were 32, 32, and 30 for A, B, and C, respectively.

Average pair agreement is the fraction of matching categories over reviewer pairs. Fleiss' kappa adjusts that observed agreement for the agreement expected from pooled category frequencies across three raters (Fleiss, 1971). We calculate it for each criterion using Pass, Fail, and N/A as nominal categories. We also report the subset on which all three reviewers supplied Pass or Fail, indicating common applicability. The pooled statistic across all 112 combinations is a descriptive summary.

The comparison reference for each document and criterion is the category selected by at least two reviewers. One condition case has three different categories and no majority. Exact categorical agreement counts identical Pass, Fail, or N/A decisions. For the binary confusion matrices, Fail is positive and Pass/N/A form the non-fail category. Thus a true positive is a document with both checker Fail and majority Fail; a false positive is checker Fail against a non-fail majority label. Precision divides shared Fail documents by all checker-Fail documents; recall divides shared Fail documents by all majority-Fail documents. These terms name the calculation against the specified document reference. Section 4.3 examines the underlying defects and rationales.

We compare the Phase-A and Phase-B majority issue sets descriptively and inspect cases appearing only in Phase B. For condition disclosure, a separate analysis uses the same ten checker-eligible documents and the labels from each reviewer. The reference labels are kept separate so that each reported score identifies the reviewers whose judgments it uses.

3.5. Rationale and rule-scope analysis

After reproducing the document-level results, we inspect every majority-Fail source-role document and all four source-role alerts. Their union contains 18 distinct documents. For each case, the ledger retains the case identifier, source excerpt, generated description, three labels and accompanying rationales, checker finding, and applicable rule pattern. We then ask whether a matching Fail label identifies the same source/description discrepancy and which distinctions a checker-Pass case requires. The topic groupings summarize the recorded rationales; the original labels remain unchanged.

For the condition comparison, we inspect all five flagged documents, including the three where the checker and majority disagree. The analysis links each source branch to the displayed formula and safeguard text, then examines the specific condition and documentary evidence cited in each rationale. This preserves both the categorical results and the evidence needed to interpret them.

4. Results

4.1. Generated descriptions and control outcomes

The checker returned every expected decision across the 18 prepared rule tests and 14 artifact-derived controls: 19 Fail, 11 Pass, and two N/A. The 30 binary controls produced no discrepancy from their expected labels. Controls that removed a denominator, repeated a safeguard heading without its explanation, changed an external parameter's name or role, removed a review-queue entry, or introduced certification wording triggered their specified rules.

Source inspection connected repeated primary-output findings to three generator operations. A role classifier misidentified comments or conversion types. Safeguard preparation failed to retain a condition or its variable consistently. A template emitted certification wording irrespective of recorded review status. Table 3 reports these occurrences and the separately verified routing behavior. The certification row describes a reproducible wording/status combination whose interpretation is examined through reviewer agreement below.

The 29-document scenario output contained one source-role finding and certification wording on all 11 unresolved documents. Its one eligible zero guard passed. The eight-document challenge output also contained one source-role finding, with certification wording on all six unresolved documents and a passing result for its one eligible zero guard. Marked conflicts retained their routing in both outputs. These comparisons show recurrence of the role and wording patterns, together with variation in the condition outcomes across the evaluated inputs.

The four FastAPI examples produced the recorded OpenAPI descriptions and adapter outcomes. Their checks and prepared substitutions exercised the mapping of parameter roles, names, constraints, and status information into the rule inputs. They contribute evidence of adapter execution and control behavior alongside the LEAP comparisons.

4.2. Reviewer agreement and document-label comparisons

Across 112 document–criterion combinations, average pair agreement was 0.634 and pooled Fleiss' kappa was 0.446. Table 4 reports the per-criterion values and label counts. The condition criterion had 12 commonly applicable documents, with kappa 0.446. For disagreement, restricting the analysis to seven commonly applicable documents changed kappa from 0.452 to −0.167. Authority judgments contained no N/A values and gave kappa −0.285.

The source-role and condition criteria remain the focus of the discrepancy analysis. The authority operationalization was set aside because its judgments showed less agreement than the frequency-based baseline. The broad disagreement criterion is reported with its applicability-sensitive agreement, while explicit-marker routing is retained as a separate

Table 3
Technical audit of current and earlier LEAP outputs. Documents, manifestations, and eligible guards are distinct counting units. Certification counts record specified wording beside unresolved status.

Recorded property	Primary output: 28 documents	Earlier output: 18 documents
Source-role rule findings	4 documents; 12 manifestations	Same 4 documents; same 12 manifestations
Condition-rule failures	5 of 10 eligible guards	4 of 9 eligible guards
Certification wording on unresolved documents	19 of 19	12 of 12
Explicit conflict routing preserved	Same 6 conflict documents; 6 of 6	Same 6 conflict documents; 6 of 6

Table 4
Observed agreement and label distributions for three reviewers. Each full-criterion row contains 84 ratings on 28 documents. Common applicability retains only documents with three Pass/Fail ratings.

Criterion	Pass	Fail	N/A	Pair agreement	Kappa	Common documents	Common-case kappa
Source-role accuracy	34	50	0	0.714	0.407	28	0.407
Condition disclosure	31	13	40	0.726	0.553	12	0.446
Representation disagreement	13	17	54	0.714	0.452	7	−0.167
Review-authority consistency	34	50	0	0.381	−0.285	28	−0.285

technical result. These are analysis decisions informed by the observed labels and examples.

Table 5 preserves the complete document-level comparisons. Source roles yield two shared Fail documents, two checker-Fail/majority-Pass documents, fourteen checker-Pass/majority-Fail documents, and ten shared Pass documents. The corresponding reference-relative precision is 50% and recall is 12.5%. For conditions, case C17 has A=N/A, B=Pass, and C=Fail and is excluded from the majority comparison. The remaining 27 cases yield precision 40% and recall 100% against that reference.

The Phase-A majority identified an issue in 18 of 28 documents. Phase B identified an issue in 23 of 28. The five additional documents, C03, C08, C11, C20, and C21, all arose from the authority criterion, with A and C assigning Fail and B assigning Pass. The additional labels therefore concern the criterion set aside after its agreement analysis.

4.3. What the source-role labels mean

The rationale inspection changes the interpretation of the two overlapping Fail documents. For C09, Flare Emission Recovery Rate, reviewers A and B discuss a mismatch between health, safety and environment (HSE) and Finance ownership, while reviewer C identifies comments described as executable statements or aliases. The checker flags the latter source-role patterns. For C18, Polymer Melt Flow Index Variance, A and B discuss a mismatch between the formula and gas-recovery variables, while C describes the comment/executable-role problem recognized by the checker. In each document, all three reviewers assign Fail, yet the checker finding corresponds to reviewer C's reason. The agreement at document level combines different judgments about that document.

The two alerts against majority-Pass labels show the converse relationship. In C12, Ethylene Production Yield, the retained Python docstring contains the phrase "as a percentage." The generator interprets the word following "as" as an output alias and publishes "Output alias: maps this expression to a in the result set." The checker and reviewer C flag the assignment of a programming role to an ordinary word in prose. In C27, On-Spec Rate, both identify the SQL conversion type `FLOAT` being called an output alias instead of `on_spec_percentage` (Figure 1). Source inspection establishes these concrete discrepancies. Their FP designation in Table 5 records disagreement with the majority category.

The fourteen majority-Fail documents without a checker alert cover several rationale topics. Table 6 includes all sixteen majority-Fail documents so the two overlapping cases remain visible in the same account. Comments described as KPI markers recur in the reviewer explanations. A KPI marker is a label used to identify a candidate calculation; the released rule's executable-role patterns do not include that documentary role. Other judgments concern ownership, formula meaning, source-system identity, and metric interpretation, which require different comparisons from the implemented role patterns.

C13 and C28 illustrate a concrete omitted documentary-role distinction. Their source comments describe a cross-border data transfer and unsigned trace records, respectively, while the published descriptions call those comments KPI markers. The checker returns Pass. Reviewers A and C identify the same marker-role problem in these cases. They provide source-grounded examples of rule-scope omissions alongside the broader interpretation differences visible elsewhere in the ledger.

Taken together, the four alerts and the full majority-Fail set explain why the aggregate counts require a rationale-level reading. The rule suite covers specified syntactic-role patterns. The review category also collects judgments about documentary roles and relationships across fields.

Table 5
Checker decisions against majority document labels. TP=true positive, FP=false positive, FN=false negative, and TN=true negative. They count document–criterion pairs, with Fail positive and Pass/N/A grouped as non-fail. Exact matches retain all three categories. The source-role rationales underlying the shared and discordant Fail labels are examined in Section 4.3.

Criterion	TP	FP	FN	TN	Precision	Recall	Exact matches	Compared documents
Source-role accuracy	2	2	14	10	0.50	0.125	12	28
Condition disclosure	2	3	0	22	0.40	1.00	22	27; C17 has no majority
Representation disagreement	0	0	4	24	Undefined	0.00	24	28
Review-authority consistency	19	0	3	6	1.00	0.864	19	28

Table 6
All 16 majority-Fail source-role documents. Votes are Code reviewers A/B/C; F=Fail and P=Pass. Rationale topics summarize the preserved explanations and may differ within a row. The last column identifies the relevant rule-scope relationship. Full source excerpts and rationales are in the case ledger.

Case	KPI document	Votes	Checker	Rationale topics	Rule-scope relationship
C01	Gross Margin Percentage	F/F/F	Pass	Formula meaning; conflict-comment role	Formula correspondence and documentary role
C05	Runtime Execution Latency	F/F/P	Pass	Metric meaning; executable-statement role	Meaning of execution versus reported latency
C06	Inventory Modification Gap	P/F/F	Pass	Comment role; metric meaning	KPI-marker and business-meaning distinctions
C09	Flare Emission Recovery Rate	F/F/F	Fail	Ownership; comment/executable role	Checker matches C's role reason; A/B assess ownership
C10	Compressor Vibration Index	F/F/F	Pass	Comment role	KPI-marker distinction outside executable-role patterns
C13	EU Data Export Volume	F/F/F	Pass	Comment role; metric meaning	KPI-marker and procedure/metric distinctions
C14	Safe Execution Latency	F/F/F	Pass	Comment role; metric meaning	KPI-marker and length/latency distinctions
C15	Signed Lineage Audit Trail	F/P/F	Pass	Comment role	KPI-marker distinction
C16	Catalyst Replacement Frequency	F/F/F	Pass	Source-system correspondence; comment role	Source-system and documentary-role comparisons
C17	Procurement Contract Compliance	F/F/F	Pass	Formula provenance; comment role	Declared/inferred formula and governance commentary
C18	Polymer Melt Flow Index Variance	F/F/F	Fail	Formula/source correspondence; comment/executable role	Checker matches C's role reason; A/B assess formula variables
C19	HSE Safety Incident Rate	F/F/F	Pass	Data-domain/owner correspondence; comment role	Domain, ownership, and documentary-role comparisons
C23	Maintenance/Operating Cost Ratio	F/F/P	Pass	Metric/formula correspondence	Variable meaning versus metric title
C24	Customer Activity Index	F/F/F	Pass	Comment role; output/calculation role	Marker and print/calculation distinctions
C25	Governed Inventory Modification	F/P/F	Pass	Comment role	Audit-control comment classified as KPI marker
C28	Audit Trail Completeness	F/F/F	Pass	Comment role; metric meaning	Unsigned-trace annotation classified as KPI marker

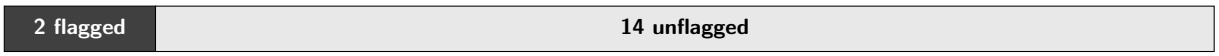
Matching or opposing document labels can therefore arise from different underlying comparisons. Figure 2 displays the label counts together with that interpretation.

4.4. Condition judgments and their underlying evidence

The condition comparison uses ten eligible documents. The checker flags C02, C10, C01, C12, and C26. Reviewers A and B have identical labels on all ten, supporting C02 and C10 and accepting the other three flagged documents. Reviewer C supports all five alerts. The remaining five documents are Pass for every reviewer and the checker. Table 7 reports both matrices with the same denominator.

The three disputed documents are Gross Margin Percentage (C01), Ethylene Production Yield (C12), and Yield Percentage (C26). Each has A=Pass, B=Pass, C=Fail, and checker=Fail. A and B describe the condition as disclosed or faithfully represented; C identifies an omitted branch or an

(a) Source-role document labels: 16 majority-Fail documents



Shared Fail labels: C09 and C18. The majority reviewers and checker identify different problems.
Two further alerts: C12 and C27. The checker identifies source-role errors on majority-Pass documents.
Document-label precision: 2/4 = 50%. Document-label recall: 2/16 = 12.5%.

(b) Condition labels: same ten documents, same five alerts

Recorded reviewer reference	Reviewer labels for the flagged documents					Alert support
	C02	C10	C01	C12	C26	
Reviewers A and B Identical labels on all ten eligible documents.	Fail	Fail	Pass	Pass	Pass	2/5 supported Precision 40%
Reviewer C Separately recorded labels on the same documents.	Fail	Fail	Fail	Fail	Fail	5/5 supported Precision 100%

The checker assigns Fail to all five displayed documents. The other five eligible documents have Pass from the checker and all three reviewers. Recall against each reference is 100%.

Figure 2: Two reference-dependent comparisons. Panel(a) counts source-role document labels; the two overlapping Fail documents have different majority and checker rationales. Panel(b) holds the ten eligible condition documents and five checker alerts fixed while comparing the recorded reviewer labels.

Table 7
Condition alerts against recorded reviewer references on the same ten eligible documents. TP=true positive, FP=false positive, FN=false negative, and TN=true negative, with Fail positive. The checker output is unchanged between rows; each row names the labels used as the reference.

Reviewer reference	TP	FP	FN	TN	Precision	Recall
Reviewers A and B (identical labels)	2	3	0	5	0.40	1.00
Reviewer C	5	0	0	5	1.00	1.00

unidentified guarded variable in the written safeguard. These Pass judgments motivated a retrospective source-visibility interpretation of the three disputed cases. The full notes show additional differences in what the reviewers observed and assessed.

In C02, Mean Time Between Failures, both A and B assign Fail despite explicitly noting the visible zero-failure guard. A questions whether the returned operating-hours value is faithfully explained; B identifies the absence of a corresponding condition in the displayed formula or safeguards. In C12, B's Pass note calls the guard "visible and explicitly safeguarded," whereas the preserved safeguard omits that branch. In C10, A's Fail rationale concerns the performance classification produced when `index >= 1.0`; the checker flags omission of the separate non-positive-baseline guard. Thus matching condition labels can also refer to different conditions, and a Pass rationale can contain an observation inconsistent with the saved document.

For C12, Listing 1 shows the retained function. Positive feedstock uses the displayed ratio, while zero or negative feedstock returns zero. The written safeguard omits that branch. For C26, the source contains a zero-total-input branch that is also omitted from the safeguard. In both cases, the source remains available and the dedicated explanatory field lacks the behavior under discussion.

```
def calculate_ethylene_yield_pct(ethylene_produced_tons: float,
      feedstock_input_tons: float) -> float:
    """Calculates ethylene production yield as a percentage of feedstock
      input."""
    if feedstock_input_tons <= 0:
        return 0.0
    return (ethylene_produced_tons / feedstock_input_tons) * 100.0
```

Listing 1: Retained source for Ethylene Production Yield (C12). The non-positive-input branch is absent from the document's safeguard field.

C01 adds a relationship between formulas and guarded variables. Its declared Gross Margin formula divides revenue minus cost by revenue. The implemented expression divides revenue by cost. The source guard protects `cost == 0`, while the written safeguard says only "zero denominator." This loses which denominator is being guarded relative to the displayed formula. Table 8 gives concrete outputs for the detailed cases.

Writing revenue as R and cost as C, the declared expression is $100(R-C)/R$ and the implemented expression is $100R/C$. For positive revenue and cost, their difference is

$$100\left(\frac{R}{C}+\frac{C}{R}-1\right)=100\left(1+\frac{(R-C)^2}{RC}\right)\geq 100.$$

Table 8
Concrete calculations illustrating the disclosure issue. Values are evaluations of the retained expressions; undefined means division by zero in the declared expression.

Calculation and inputs	Declared (%)	Implemented (%)
Yield: output 20, input 100	20	20
Yield: output 20, input 0	Undefined	0
Yield: output 20, input −10	−200	0
Gross margin: revenue 100, cost 50	50	200
Gross margin: revenue 100, cost 0	100	0
Gross margin: revenue 0, cost 50	Undefined	0

The expressions therefore differ by at least 100 percentage points on that domain. The document retains both expressions and routes their explicit disagreement for review. Its preserved conflict routing and its ambiguous safeguard concern separate fields and checks.

The condition precision values follow directly from the recorded labels: two of five alerts are Fail under A/B's reference and five of five are Fail under C's reference. The 100% row records agreement with C on this ten-document set. The three changed reference labels account for the numerical difference; the notes establish differences in assessment targets and observations about the explanation.

5. Discussion

5.1. What the study establishes

The findings identify three distinct evidence levels in evaluating generated-documentation checks. Prepared controls establish the rule's specified behavior on those inputs. Document-level comparisons establish how often its category matches a named reference category. Source and rationale inspection establishes what each assessment is actually identifying. The LEAP evidence shows how these levels can diverge within the same evaluation.

The source-role results are especially informative because the apparent agreements and disagreements both require inspection. C09 and C18 count as shared Fail documents even though the majority rationales concern different fields from the checker finding. C12 and C27 count as reference-relative false positives even though their source excerpts establish the particular role errors flagged. C13 and C28 identify omitted documentary-role distinctions that the implemented patterns leave unchecked. These observations answer RQ3 through separate mechanisms rather than one pooled explanation for all fourteen checker-Pass/majority-Fail documents.

The condition result answers RQ4 through the recorded reviewer references. The three changed labels explain the difference in precision, while the notes reveal several reasons for disagreement. The C02 Fail judgments require explanatory information beyond a visible branch; the C12 Pass rationale includes an incorrect observation about the safeguard; and C10 contains matching Fail labels for different conditions. The source-role and condition analyses therefore support the same evaluation practice: inspect the cited discrepancy and document evidence alongside the category.

RQ1 supplies the mechanism context: the generator's role classifier, safeguard preparation, and status-independent wording recur in the archived outputs. RQ2 supplies the categorical record: per-criterion agreement differs, applicability affects the disagreement subset, and the authority criterion has negative chance-adjusted agreement. Together, the four questions explain both the generated descriptions and the assessment procedure used to judge them.

5.2. Relationship to established verification and evaluation

The case evidence complements controlled-language and contract-based checking (Kuhn and Bergel, 2014; Lozano et al., 2015). Those approaches make a property explicit enough to execute. In the present evaluation, the source-role label was applied to a wider set of relationships than the implemented patterns. The observed mismatch makes the distinction between a criterion and its rule operational: an evaluator can identify the source relationship required by a judgment and check whether that relationship belongs to the rule's supported vocabulary.

The rationale analysis also connects with defect-matching evaluations such as Habib and Pradel (2018). A document can contain several concerns, so categorical co-occurrence is a coarser unit than a matched source-level discrepancy. Our preserved matrix is useful for comparing document decisions; the case ledger explains the mechanisms behind it. This relationship gives a concrete reason to retain the affected source span, document field, and written rationale when collecting labels.

Behavioral documentation research clarifies the condition result (Zhou et al., 2017; Yang et al., 2026; Eghbali et al., 2026). A condition may be available in source while absent from the explanation intended to communicate it. The evaluation must specify which representation carries the obligation. This study supplies inspectable instances of that difference, including the wrong-variable safeguard in Gross Margin and omitted branches in the two Yield documents. Its contribution to evaluation research is the documented relationship between those instances and the reference-relative scores.

5.3. An evidence structure for subsequent checking studies

The case analysis supports recording six linked items for each decision: the documentation property, source excerpt, generated field, rule reason, reviewer category, and reviewer rationale. These items let an evaluator distinguish an omitted rule pattern, a different assessment target, and a genuinely matching discrepancy. Their purpose follows from the observed cases: the same Fail label at C09/C18 concealed different problems, while source inspection explained the discordant alerts at C12/C27.

Subsequent studies can specify the required publication layer before collecting labels and define whether evaluation counts documents or individual discrepancies. Control tests can continue to verify implemented behavior, with a separate source-grounded reference for empirical comparisons. The source-role ledger identifies concrete extensions to investigate, including documentary-marker classification and relationships between titles, formulas, variables, and ownership. The condition cases identify variable binding and branch disclosure as explicit comparison requirements.

6. Limitations

The 28 primary documents came from one author-developed generator and a fixed sample-project output. Shared rules produced repeated descriptions, so document counts are not independent generator observations. The historical, scenario, and challenge outputs extend within-system evidence. The FastAPI cases establish execution on another format and are not an independently labelled external accuracy sample. The case selection supports a detailed diagnosis of these artifacts rather than a population estimate of documentation quality.

The three reviewers' backgrounds and relevant experience were not recorded in the returned forms. Their majority categories are a comparison reference, not adjudicated defect labels. The source-role criterion admitted heterogeneous rationale topics; the retrospective ledger organizes those topics without replacing the original judgments. Positive kappa measures observed category agreement and does not validate the correctness of every rationale. Overall kappa pools different criteria, while common-applicability analyses change the evaluated subset. No inference to a new reviewer population is made.

Archived guidance contains both two-reviewer and three-reviewer versions and differing agreement-statistic instructions. The completed study used three forms, Fleiss' kappa, and majority labels, without the planned adjudication. Both phase instructions were in one packet and the phase order was fixed, so the change from 18 to 23 flagged documents cannot identify a checklist effect separately from familiarity or prior access to the criteria. All five additional cases arose from the authority criterion.

The condition comparison uses recorded reviewer references rather than prospectively assigned disclosure rubrics. The notes do not establish two definitions consistently applied across the eligible set: A/B also require explanatory information beyond source visibility, and some recorded observations conflict with the saved text. Reviewer identity, assessment target, and observation are inseparable in this retrospective comparison. The 100% row uses one reviewer's labels on ten eligible documents. Establishing defect-level accuracy or a causal effect of a revised rubric requires a separately defined and adjudicated reference exercise.

Generator mechanisms were traced by source inspection; a prospective repair-and-retest experiment was not performed. The source-role rules cover selected patterns, the routing check covers explicit conflict markers, and the condition rule covers specified zero/non-positive guards. Near-zero tolerances and other function-domain restrictions require additional rules. Human comprehension, correction time, and collaborative benefit were not measured. The source-level reanalysis establishes particular discrepancies and evaluation relationships without estimating these downstream outcomes.

7. Conclusion

This study explains how rule scope, evaluation unit, and reference criteria shape checking results for generated KPI documentation. The checker reproduced all 32 prepared control outcomes. Against majority source-role document labels it flagged two of 16 Fail documents, giving 12.5% reference-relative recall. Inspection showed that those two shared labels concerned different majority and checker rationales, while two majority-discordant alerts identified concrete source-role errors. The full ledger separates omitted documentary-role patterns from broader differences in what reviewers assessed.

On ten eligible condition documents, the same five alerts had precision 40% against the identical labels of reviewers A and B and 100% against reviewer C's labels. Three changed reference labels account for that difference. Their rationales show differences in the condition assessed and observations about its explanation. Together with the preserved sources, generator excerpts, and reviewer rationales, these findings provide a reproducible empirical account of why correct control decisions, matching document categories, and agreement on the same discrepancy must be examined separately when assessing documentation checkers.

Data and artifact availability

The complete replication package is linked at `https://github.com/Meshaal-Mouawad/Paper_2_JSS`. It contains the 28 primary documents, earlier and alternative-input outputs, source and generator excerpts, checking rules, 32 prepared controls and expected outcomes, original reviewer prompts and instructions, all 336 original ratings, reviewer rationales, case-level comparisons, analysis scripts, and verification records. The repository explains how each reported result can be reproduced and distinguishes archived output checks from complete generator execution.

Declaration of competing interest

The authors declare that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this manuscript, the authors used Elsevier LeapSpace and ChatGPT Work to support

literature discovery, source checking, manuscript organization, language refinement, and consistency checks. ChatGPT Work also assisted with verification scripts and reanalysis of preserved study records. The authors reviewed and edited the resulting material and take full responsibility for the content of this article.

References

Aghajani, E., Nagy, C., Vega-Márquez, O.L., Linares-Vásquez, M., Moreno, L., Bavota, G., Lanza, M., 2019. Software documentation issues unveiled, in: Proceedings of the IEEE/ACM 41st International Conference on Software Engineering, IEEE. pp. 1199–1210. doi:10.1109/ICSE.2019.00122.

van Deursen, A., Moonen, L., 2006. Documenting software systems using types. Science of Computer Programming 60, 205–220. doi:10.1016/j.scico.2005.10.006.

Eghbali, A., Liu, Z., Pradel, M., 2026. Natural language-focused software engineering via code-documentation equivalence. Proceedings of the ACM on Software Engineering 3. doi:10.1145/3808211.

FastAPI Project, 2026. FastAPI tutorial source examples. GitHub repository, commit 33d411dbc3236275dd64d200bfe18d5d60a49b2e. URL: https://github.com/fastapi/fastapi/tree/33d411dbc3236275dd64d200bfe18d5d60a49b2e/docs_src.

Fleiss, J.L., 1971. Measuring nominal scale agreement among many raters. Psychological Bulletin 76, 378–382. doi:10.1037/h0031619.

Habib, A., Pradel, M., 2018. How many of all bugs do we find? a study of static bug detectors, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ACM. pp. 317–328. doi:10.1145/3238147.3238213.

Knuth, D.E., 1984. Literate programming. The Computer Journal 27, 97–111. doi:10.1093/comjnl/27.2.97.

Kuhn, T., Bergel, A., 2014. Verifiable source code documentation in controlled natural language. Science of Computer Programming 96, 121–140. doi:10.1016/j.scico.2014.01.002.

Lozano, A., Mens, K., Kellens, A., 2015. Usage contracts: Offering immediate feedback on violations of structural source-code regularities. Science of Computer Programming 105, 73–91. doi:10.1016/j.scico.2015.01.004.

OpenAPI Initiative, 2025. OpenAPI specification, version 3.1.2. OpenAPI Initiative specification. URL: https://spec.openapis.org/oas/v3.1.2.html. accessed 31 October 2026.

Roy, D., Fakhoury, S., Arnaoudova, V., 2021. Reassessing automatic evaluation metrics for code summarization tasks, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM. doi:10.1145/3468264.3468588.

Shi, E., Wang, Y., Du, L., Chen, J., Han, S., Zhang, H., Zhang, D., Sun, H., 2022. On the evaluation of neural code summarization, in: Proceedings of the 44th International Conference on Software Engineering, ACM. pp. 1597–1608. doi:10.1145/3510003.3510060.

Wen, F., Nagy, C., Bavota, G., Lanza, M., 2019. A large-scale empirical study on code-comment inconsistencies, in: Proceedings of the IEEE/ACM 27th International Conference on Program Comprehension, IEEE. pp. 53–64. doi:10.1109/ICPC.2019.00019.

Xu, Z., Guo, S., Wang, Y., Chen, R., Li, H., Li, X., Jiang, H., 2024. Code comment inconsistency detection based on confidence learning. IEEE Transactions on Software Engineering 50, 598–617. doi:10.1109/TSE.2024.3358489.

Yang, Y., Li, Q., Bao, T., Li, Q., Wang, W., 2026. OCL2NL: Automatic generation of natural language descriptions from OCL contracts. Science of Computer Programming 254, 103513. doi:10.1016/j.scico.2026.103513.

Zhong, H., Su, Z., 2013. Detecting API documentation errors, in: Proceedings of the ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages and Applications, ACM. pp. 803–816. doi:10.1145/2509136.2509523.

Zhou, Y., Gu, R., Chen, T., Huang, Z., Panichella, S., Gall, H.C., 2017. Analyzing APIs documentation and code to detect directive defects, in: Proceedings of the IEEE/ACM 39th International Conference on Software Engineering, IEEE. pp. 27–37. doi:10.1109/ICSE.2017.11.